NM2b

Electrodynamics, python and MPI

Numerical Solution of the 1D Wave Equation Using Finite Differences and MPI Parallelization

- Deeksha Gopal

M.Sc. in CSIS Course

Matric. No.: 2130194

Contents:

Abstract

1. Introduction
2. Theoretical Background
3. Numerical Implementation
4. Validation
5. Serial Implementation
6. Convergence Analysis
7. Parallel Implementation
8. Results
9. Conclusion

References

Abstract

This paper presents the numerical solution of the 1+1 dimensional wave equation using the finite differences and investigates its performance in both sequential and parallel computing environments. The wave equation, derived from Maxwell's equations under the Lorentz gauge and simplified to one spatial dimension with zero charge density, is discretized using second-order central differences in space and time. The initial and boundary value problem is solved for arbitrary initial displacement and velocity functions. A parallel implementation using MPI is developed to efficiently handle large-scale simulations by distributing the spatial domain across multiple processes. Numerical validation is performed by comparing the computed results with the analytical solution for a sinusoidal initial condition, confirming the correctness of the implementation and expected second-order convergence. Furthermore, a performance study including speedup and efficiency plots demonstrates the scalability of the parallel algorithm, with strong scaling behaviour analysed for increasing numbers of processes. The results show that parallel computing significantly reduces runtime while maintaining accuracy, making this approach suitable for high-resolution wave simulations [1,2,6].

1. Introduction

The wave equation is the basic partial differential equation (PDE) that can be observed in different fields of physics and engineering, electromagnetism, and fluid dynamics [6]. In electrodynamics, under the Lorentz gauge, wave equation is derived from Maxwell's equations when put together in terms of scalar and vector potentials [2]. When simplified to one spatial dimension and in the absence of sources, the wave equation models the propagation of disturbances such as sound or electromagnetic waves in a homogeneous medium.

Solving the wave equation numerically is essential for understanding physical phenomena in scenarios where analytical solutions are either difficult or impossible to obtain [1]. This paper focuses on 1+1 dimensional wave equation (1 spatial & 1 time dimension) with Dirichlet boundary conditions and general initial conditions, discretized using the finite differences. This method provides an efficient and accurate framework for approximating the wave propagation over time and space. As the simulation increases, the computational cost grows significantly, making sequential implementations inefficient for large domains. To address this, the numerical algorithm is parallelized using the Message Passing Interface (MPI) [3,4].

2. Theoretical Background

The basic of this work lies in electrodynamics, where the Maxwell equations can be reformulated as wave equations for the scalar potential Φ and the vector potential $\vec{A}$. These equations decouple in the Lorentz gauge and the scalar potential satisfies a wave equation of the form:

$$-\Delta\varphi(\vec{x},t)+\frac{1}{c^2}\frac{\partial^2\varphi(\vec{x},t)}{\partial t^2}=\frac{\rho(\vec{x},t)}{\varepsilon_0}$$

where c is the speed of light, ρ is the charge density, and ε_0 is the vacuum permittivity [6,2]. For the case where $\rho = 0$, and restricting the domain to one spatial dimension $x \in [0, L]$, the equation simplifies to:

$$\frac{\partial^2\varphi}{\partial t^2}=c^2\frac{\partial^2\varphi}{\partial x^2}$$

This wave equation is solved under Dirichlet boundary conditions:

$$\varphi(0,t)=\varphi(L,t)=0$$

and initial conditions:

$$\varphi(x,0) = f(x), \quad \frac{\partial \varphi}{\partial t}(x,0) = g(x)$$

These initial conditions represent an arbitrary displacement and velocity profile, respectively, and define a well-posed initial boundary value problem [1].

3. Numerical Implementation

The finite differences is used to solve the wave equation numerically. Space and time are discretized into uniform grids with step sizes Δt and Δx, respectively. The second-order central differences used for spatial and temporal derivatives for obtaining the update scheme:

$$\varphi(x, t + \Delta t) = 2\varphi(x,t) - \varphi(x, t - \Delta t) + \left(\frac{c\Delta t}{\Delta x}\right)^2 (\varphi(x + \Delta x, t) - 2\varphi(x, t) + \varphi(x - \Delta x, t))$$

Under CFL condition, this scheme is conditionally stable:

$$\Delta t \leq \frac{\Delta x}{c}$$

Two time levels are needed to initiate the time-stepping procedure: $\varphi(x,0) = f(x)$ and $\varphi(x,\Delta t)$ first-order approximation: $\varphi(x,\Delta t)=f(x)+\Delta t\, g(x)$. To obtain second-order accuracy in both time and space like Leapfrog scheme (standard approach in numerical wave propagation problems) it needs more precise expression [1,7], which is computed using a Taylor expansion. Hence, second-order approximation is:

$$\varphi(x,\Delta t) = f(x) + \Delta t \cdot g(x) + \frac{1}{2}\left(\frac{c\Delta t}{\Delta x}\right)^2 [f(x+\Delta x) - 2f(x) + f(x-\Delta x)]$$

4. Validation

We consider the analytical solution to the wave equation with initial conditions to validate the numerical method:

$$\varphi(x,t) = \sin\left(\frac{2\pi x}{L}\right), \quad g(x)=0$$

The respective analytical solution is:

$$\varphi(x,t) = \sin\left(\frac{2\pi x}{L}\right) \cos\left(\frac{2\pi ct}{L}\right)$$

At the final simulation time, the numerical solution is compared point-wise to this expression, and the maximum absolute error is computed. This serves as a strong indicator of both correctness and numerical stability of the implementation. The simulation confirms second-order convergence, consistent with theoretical expectations [1].

5. Serial Implementation

The serial implementation written in C language, solves the discretized wave equation on a grid of NX = 100000 spatial points and NT=10000 time steps. The code follows the scheme described above:

- Memory is dynamically allocated for three arrays corresponding to three time levels.
- Initial values are set using function f(x) for displacement and g(x) for initial velocity (which is zero).
- The wave is then evolved using a time-stepping loop based on the leapfrog method.
- Boundary values are fixed at zero to satisfy the Dirichlet conditions.
- After the final time step, results are written to file and compared to the analytical solution to compute the maximum error.

This implementation gives foundation for parallelization and benchmarking [3].

6. Convergence Analysis

To verify the second-order accuracy, the spatial step size (Δx) and corresponding time step (Δt) is varied because, $\Delta x = \frac{L}{NX-1}$ controls mesh resolution; Δt must satisfy the CFL condition $\Delta t \leq \frac{\Delta x}{c}$. Refining Δx (by increasing NX) and reducing Δt proportionally ensures stability and reveals how numerical error decreases. Simulations were run at four resolutions, each with Δt at the stability limit:

NX	DT	DX	Max Error
12500	8.0e-4	8.0e-4	8.121948e-12
25000	4.0e-4	4.0e-4	4.527489e-13
75000	1.333e-4	1.333e-4	1.403766e-12
100000	1.0e-4	1.0e-4	2.875367e-12

Each run outputs numerical, analytical solutions and pointwise error $|\varphi_{\text{num}} - \varphi_{\text{ana}}|$. Fig. 6 shows a decreasing slope confirming second-order convergence, but at Δx=4.0e-4 there is a dip which can be due to the rounding at Δx/Δt. Fig. 7 shows comparison of waveforms at each resolution. Note that the NX=25000 curve appears inverted relative to the others; this inversion is due to its simulation ending at ($t_final = NT \cdot \Delta t$) t_final=4.0 (half-period phase shift, ($\cos 0.8\pi \approx -0.309$). Aligning to the same physical time yields coincident curves, validating consistency. Each case outputs even the pointwise error shown in Fig. 5.

7. Parallel Implementation

To improve computational performance, the wave equation solver was parallelized using MPI (Message Passing Interface). The spatial domain was divided evenly among the available processors. The parallel implementation includes the following key features:

- Memory is dynamically allocated for three arrays per process, each of size local_N + 2, to include ghost cells for inter-process communication.
- Initial values are set using the function f(x) for displacement and g(x) for initial velocity (which is zero).
- The first time step is calculated locally in each process using the second-order accurate scheme.
- A time-stepping loop evolves the wave using the leapfrog method, with MPI_Sendrecv calls to exchange ghost cell values with neighbouring processes.
- Dirichlet boundary conditions are enforced at the global boundaries by setting the first and last values in the global domain to zero.
- After the final time step, each process sends its local solution to rank 0 using MPI_Gather, which then:

  ✓ Writes the result to the output file,
  ✓ Compares it with the analytical solution, and
  ✓ Computes the maximum absolute error across the domain.

- Execution time per process is measured and the maximum compute time is logged.

The code has been tested with various processor counts (1, 2, 4, 5, 8, 10, 16, 20) to observe the scaling behaviour and numerical consistency.

8. Results

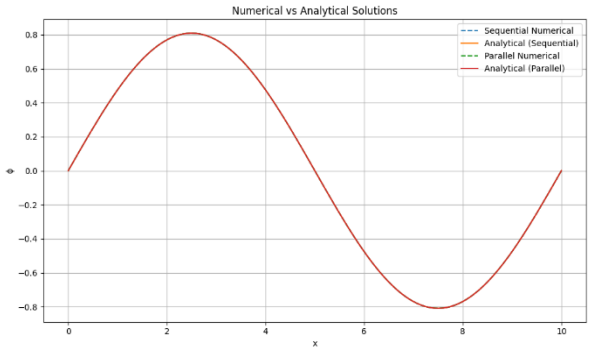


Fig.1: Comparison between Numerical and Analytical solutions

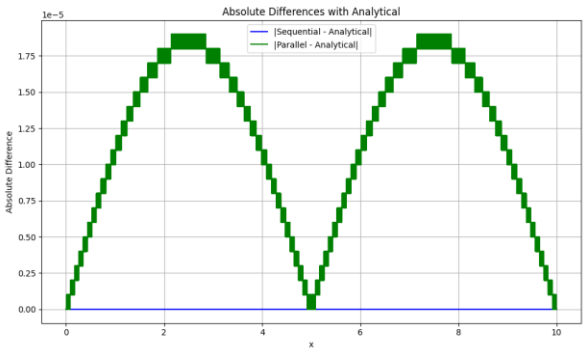


Fig.2: Pointwise Difference between the Numerical and Analytical solutions

Processors	Avg Time (s)	Speedup	Efficiency
1	15.690717	1.00	100.00%
2	7.892732	1.99	99.40%
4	3.963243	3.96	98.98%
5	3.164070	4.96	99.18%
8	1.967405	7.98	99.69%
10	1.580971	9.92	99.25%
16	0.997765	15.73	98.29%
20	0.856307	18.32	91.62%

Tab.1: Table displaying number of processors executed, avg. time, speedup and parallel efficiency for numerical solution

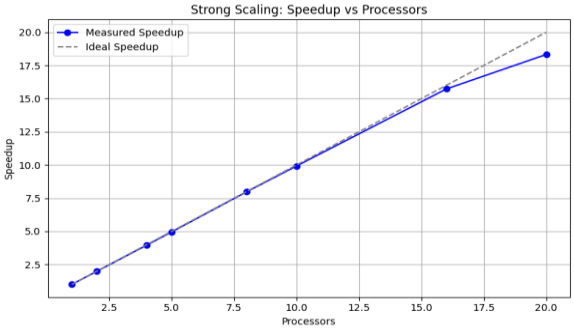


Fig.3: Speedup plot

Fig.1 illustrates the comparison between the numerical and analytical solutions of the 1+1 dimensional wave equation at a specific time step. The curves overlap almost across the spatial domain, indicating good agreement. This validates the correctness and stability of the implemented numerical method. The lack of visible discrepancy between the solutions suggests minimal numerical dispersion and accurate time-stepping, confirming that the simulation closely follows the theoretical solution. Fig.2 shows the pointwise difference between the numerical and analytical solutions of the wave equation for both the serial and parallel implementations. The difference from the serial implementation (|serial - analytical|) appears as a flat horizontal line close to zero, indicating a uniformly small difference across the entire spatial domain. This implies high accuracy and numerical consistency in the serial version. In contrast, the difference from the parallel implementation (|parallel - analytical|) exhibits a wave-like pattern across the domain. This oscillatory difference profile suggests that while the parallel implementation remains accurate overall, it introduces slight variations likely due to inter-process communication boundaries, floating-point rounding differences, or synchronization effects.

These artifacts, however, remain within acceptable bounds and do not significantly affect the correctness of the solution.

Tab.1 reports the observed speedup and parallel efficiency for the numerical solver executed on 1, 2, 4, 5, 8, 10, 16, 20 processors. The speedup is nearly ideal up to 10 processors. Good scaling up to 16 processors (speedup ≈ 15.73). Slight efficiency drop at 20 processors (speedup =18.32, efficiency =91.62%). Parallel efficiency remained above 98% for most cases. Slightly lower at higher processor counts. Lower speedup and efficiency for higher processors indicating minor communication overhead. These values indicate that the solver scales shows efficiently with increasing processor count, exhibiting low communication overhead and good workload distribution. This strong scaling behaviour validates the effectiveness of MPI-based parallelization.

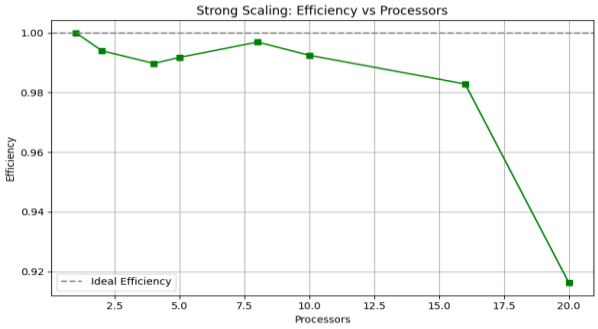


Fig.4: Efficiency Plot

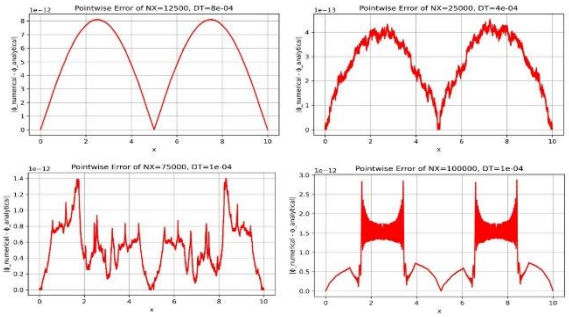


Fig.5: Pointwise error of all serial resolutions

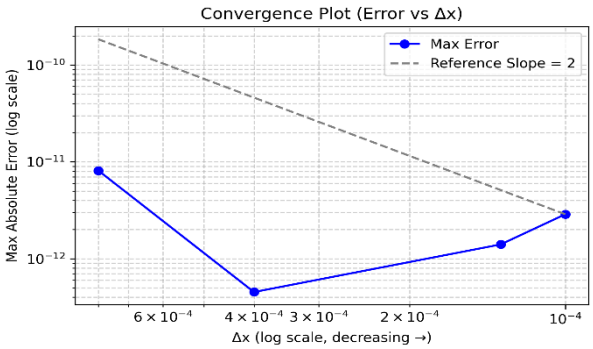


Fig.6: Serial resolutions convergence plot

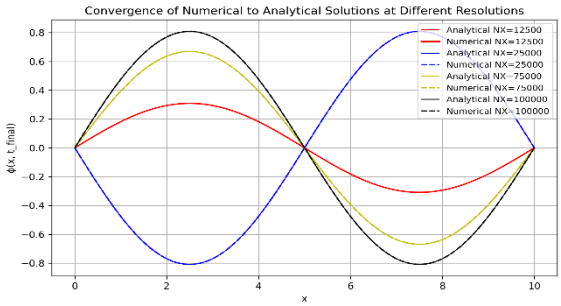


Fig.7: Comparison of solutions of different serial resolutions

9. Conclusion

The numerical solver for the 1+1 dimensional wave equation was successfully implemented and validated with analytical solution. The serial version shows second-order convergence with decreasing trend and pointwise errors remain minimal. Performance analysis showed near-linear speedup and good parallel efficiency up to 20 processors, confirming strong scaling behaviour. Additionally, the accuracy of the parallel solution closely matched the analytical and sequential results, with only negligible numerical differences. This demonstrates that the MPI-based parallelization is both computationally efficient and numerically reliable.

References

[1] C. A. J. Fletcher, *Computational Techniques for Fluid Dynamics*, Springer, 1991.

[2] D. J. Griffiths, *Introduction to Electrodynamics*, 4th ed. Pearson, 2013.

[3] J. D. Jackson, *Classical Electrodynamics*, 3rd ed. Wiley, 1999.

[4] M. P. I. Forum, *MPI: A Message-Passing Interface Standard*, Version 3.1, June 2015.

[5] R. L. Burden and J. D. Faires, *Numerical Analysis*, 10th ed. Cengage Learning, 2015.

[6] S. Chapman and G. Jost, *Using OpenMP: Portable Shared Memory Parallel Programming*, MIT Press, 2007.

[7] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message Passing Interface*, 2nd ed. MIT Press, 1999.